

```
def delete_at_first(self):
```

```
    if self.head is None:
```

```
        print('No Nodes to delete')
```

```
    elif self.length() == 1:
```

```
        self.head = None
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next != self.head:
```

```
            temp = temp.next
```

```
        temp.next = self.head.next
```

```
        self.head.next.prev = temp
```

```
        self.head = self.head.next
```

```
def search_the_ele(self, val):
```

```
    if self.head is None:
```

```
        print('No element in DLL')
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next != self.head:
```

```
            if val == temp.data:
```

```
                print(val, 'Value found')
```

```
                break
```

```
            temp = temp.next
```

```
        else: if temp == self.head:
```

```
            print(val, 'Value not found')
```

```
            break
```



```
def display_nodes_prev_and_next(self, val):
```

```
    prev_data = None
```

```
    next_data = None
```

```
    if self.head == None:
```

```
        print('No ele in CDLL')
```

```
    temp = self.head
```

```
    while temp:
```

```
        if temp.data == val:
```

```
            next_data = temp.next.data
```

```
            prev_data = temp.prev.data
```

```
            break
```

```
        temp = temp.next
```

```
    if temp == self.head:
```

```
        print(f'{val} is not present in CDLL')
```

```
        break
```

```
    print(f'Previous Data: {prev_data} \n Next Data: {next_data}')
```


Bucket Sort:-

- It is a unique sorting algo helps while sorting huge amount of element. This algo takes extra space and extra time while sorting.

Algorithm:-

- Step 1
- find the total no of bucket to create.
~~var~~ = import math
var = round(math.sqrt(len(l1)))
- 2.
- Create a total no of bucket based on above value.
- 3.
- iterate through each element and find the bucket no to insert.
$$\text{math.ceil}(\text{value} * \text{total buckets}) / \text{max_ele_in_list})$$
- 4.
- Sort each buckets with the help of another sorting technique.
- 5.
- Merge all the buckets.

eg:-

```
import math
```

```
def bucket_sort(l1):
```

```
    # find total bucket to create
```

```
    total_bucket = round(math.sqrt(len(l1)))
```

```
    print(total_bucket)
```

```
    # Create total no of bucket.
```

```
    buckets = []
```

```
    for _ in range(total_bucket):
```

```
        buckets.append([])
```

```
    print(buckets)
```



```
# find the bucket no for each element.
max_ele = max(l1)
for i in range(len(l1)):
    idx = math.ceil((l1[i] * total_buckets) / max_ele)
    buckets[idx-1].append(l1[i])
print(buckets)
```

```
# Sort each bucket
for i in range(total_buckets):
    buckets[i] = bubble_sort(buckets[i])
print(buckets)
```

merge all the buckets:-

```
k = 0
for i in range(total_buckets):
    for j in range(len(buckets[i])):
        l1[k] = buckets[i][j]
        k += 1
```

l1 = [9, 3, 1, 5, 7, 2, 6, 4, 8]

print('Before sorting ->', l1)

bucket_sort(l1)

print('After sorting ->', l1)

[[] [] []]
0 1 2

round(4.7) 5

round(4.3) 4

$$(7 * 3) \mid 9$$

$$21 \mid 9 \Rightarrow 2.3 \dots \Rightarrow \underline{3}$$

— / — / —

- ### Partition Algorithm:-

- | | | | | | | | | |
|--------------|---|---|---|---|---|---|--------------|------------------|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 3 | 5 | 8 | 1 | 2 | 9 | 4 | 7 | 6 |
| $i \uparrow$ | | | | | | | $j \uparrow$ | Pivot $\uparrow$ |

- $$i^{\text{th}} \text{ index} < j^{\text{th}} \text{ index}$$

- i^{th} value $<$ Pivot value
 $i++$

- if value > pivot value
j--

//_

When both i and j stops moving check
 i^{th} index is less the j^{th} index or not,
if it is true Swap i^{th} and j^{th} value.

$i^{\text{th}} \text{ idx} < j^{\text{th}} \text{ idx}$
Swap i^{th} and j^{th} value

Step 6 Go to Step 2 again

7 When outer loop is stops check i^{th} value
is greater than pivot value or not, if it
is true swap i^{th} value and pivot value.
and return i^{th} index. $\rightarrow$ Partition position.

0	1	2	3	4	5	6	7	8
3	5	4	1	2	6	8	7	9
				j	i			

```
def partition (start, end, l1):
```

```
    i = start                # index  
    j = end - 1             # index  
    pivot = l1[end]         # value
```

```
    while i < j:
```

```
        while l1[i] < pivot:  
            i += 1
```

```
        while l1[j] > pivot:  
            j -= 1
```

```
    if i < j:
```

```
        l1[i], l1[j] = l1[j], l1[i]
```

```
    if l1[i] > pivot:
```



```
l1[i], l1[end] = l1[end], l1[i]  
return i
```

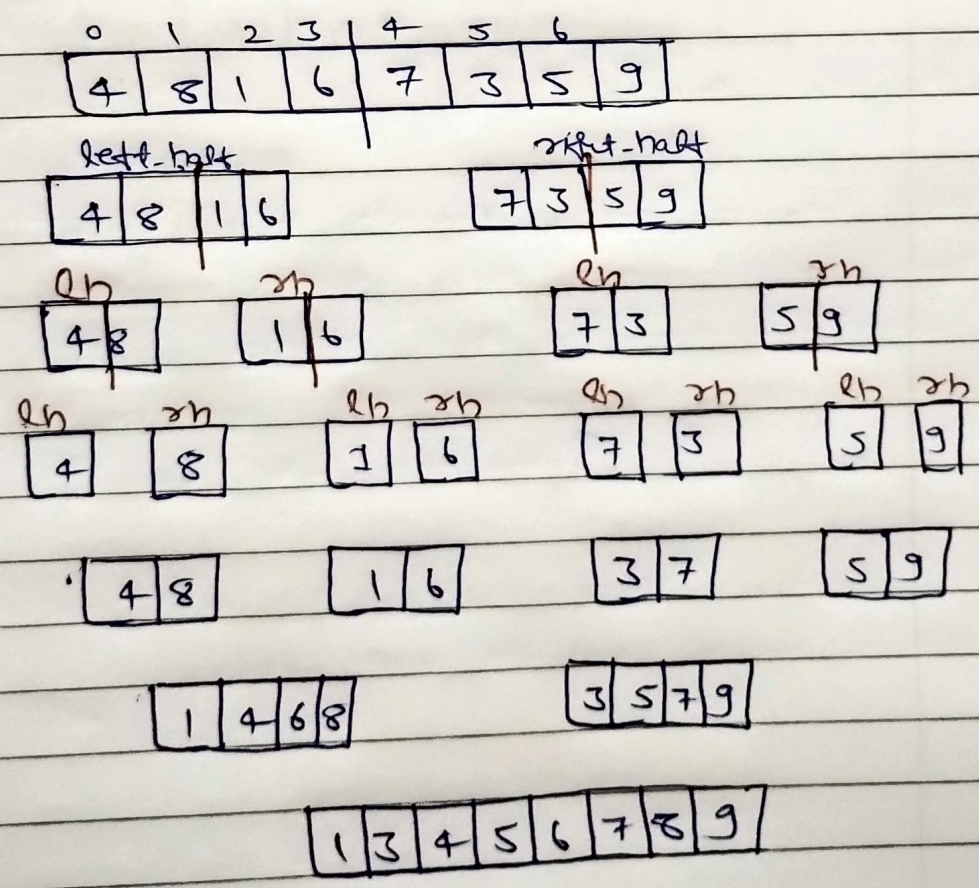
```
def quick-sort(start, end, l1):  
    if start < end:  
        pi = partition(start, end, l1):  
        quick-sort(start, pi-1, l1)  
        quick-sort(pi+1, end, l1)
```

```
l1 = [3, 5, 8, 1, 2, 9, 4, 7, 6]  
print('Before Sorting', l1)  
quick-sort(0, len(l1)-1, l1)  
print('After Sorting', l1)
```


Merge Sort:-

- It is a advance sorting technique which helps us to sort the element with the help of divide and conquer rule.
- In this first need to divide the element into two equal half and continue the process untill list can't be divided two equal half.
- When the dividing is stops start merging the list by comparing left and right list element.

eg:-



//_

```
def merge_sort(l1):
```

```
    if len(l1) > 1:
```

```
        mid = len(l1) // 2
```

```
        left_half = merge_sort l1[:mid]
```

```
        right_half = l1[mid:]
```

```
        merge_sort(left_half)
```

```
        merge_sort(right_half)
```

```
    # Start merging
```

```
    i, j, k = 0, 0, 0
```

```
    while i < len(left_half) and j < len(right_half):
```

```
        if left_half[i] < right_half[j]:
```

```
            l1[k] = left_half[i]
```

```
            i += 1
```

```
        else:
```

```
            l1[k] = right_half[j]
```

```
            j += 1
```

```
        k += 1
```

```
    # Check any of the element is remaining  
    in both the list.
```

```
    while i < len(left_half):
```

```
        l1[k] = left_half[i]
```

```
        i += 1
```

```
        k += 1
```

```
    while j < len(right_half):
```

```
        l1[k] = right_half[j]
```

```
        j += 1
```

```
        k += 1
```

```
l1 = [4, 8, 1, 6, 7, 3, 5, 9]
```

```
Print('Before Sorting -> l1')
```

```
merge_sort(l1)
```

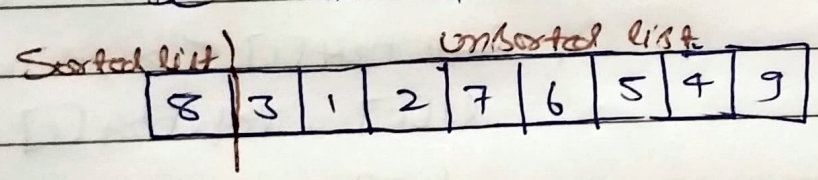
```
Print('After Sorting -> l1')
```


Insertion Sort:-

- It is a sorting algorithm which is efficient for both huge amount of element and limited no of element

Algorithm:-

- Consider given list into two parts like Sorted and Unsorted list, In that first element must be inserted list and remaining all the element must be inserted list.



- Start iteration by comparing both the list elements
- Take first element from unsorted list and compare with last element in sorted list like sorted list element is greater than unsorted list element, if it is true swap the elements.
- The above process should continue till unsorted list element becomes empty.